

Spring Boot Learning - Exercises

Current status

Current block: **Block 2 - Validation and HTTP responses**

Completed exercises: **01 to 10**

Current exercise: **Exercise 11 - Reusable validation with private methods**

Exercise index

Exercise	Topic	Method	URL	Status
01	Basic endpoint	GET	http://localhost:8080/exercise01/hello	Done
02	Path variable	GET	http://localhost:8080/exercise02/hello/{name}	Done
03	Request param	GET	http://localhost:8080/exercise03/hello? name=Angel	Done
04	JSON response	GET	http://localhost:8080/exercise04/student	Done
05	JSON + PathVariable	GET	http://localhost:8080/exercise05/student/{name}	Done
06	JSON list	GET	http://localhost:8080/exercise06/students	Done
07	Filter by query param	GET	http://localhost:8080/exercise07/students? role=developer	Done
08	POST + RequestBody	POST	http://localhost:8080/exercise08/students	Done
09	ResponseEntity	POST	http://localhost:8080/exercise09/students	Done
10	Basic validation with POST	POST	http://localhost:8080/exercise10/students	Done
11	Reusable validation with private methods	POST	http://localhost:8080/exercise11/students	Done

Exercise 01 - Basic endpoint

Purpose:

...

URL:

http://localhost:8080/exercise01/hello

HTTP method:

GET

Main concepts:

- `@RestController`
- `@GetMapping`
- Basic endpoint

File:

`src/main/java/.../HelloController.java`

Status:

Done

Exercise 02 - Path variable

Purpose:

This exercise introduces the use of path variables to capture dynamic values directly from the URL.

URL:

`http://localhost:8080/exercise02/hello/{name}`

HTTP method:

GET

Main concepts:

- `@GetMapping`
- `@PathVariable`
- Dynamic URL values
- Basic REST parameter handling

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise02/PathVariableController.java`

Status:

Done

Exercise 03 - Request param

Purpose:

This exercise introduces the use of query parameters to receive optional or specific values from the URL.

URL:

`http://localhost:8080/exercise03/hello?name=Angel`

HTTP method:

GET

Main concepts:

- `@GetMapping`

- `@RequestParam`
- Query parameters
- Basic parameter handling in HTTP requests

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise03/RequestParamController.java`

Status:

Done

Exercise 04 - JSON response

Purpose:

This exercise introduces how Spring Boot automatically converts a Java object into a JSON response.

URL:

`http://localhost:8080/exercise04/student`

HTTP method:

GET

Main concepts:

- `@RestController`
- `@GetMapping`
- JSON response
- Java object serialization

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise04/JsonController.java`

Status:

Done

Exercise 05 - JSON with PathVariable

Purpose:

This exercise combines a JSON response with a path variable to return student data based on a value received from the URL.

URL:

`http://localhost:8080/exercise05/students/{id}`

HTTP method:

GET

Main concepts:

- `@GetMapping`

- `@PathVariable`
- JSON response
- Dynamic resource lookup by ID

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise05/StudentByIdController.java`

Status:

Done

Exercise 06 - JSON list response

Purpose:

This exercise introduces how Spring Boot can return a list of Java objects as a JSON array.

URL:

`http://localhost:8080/exercise06/students`

HTTP method:

GET

Main concepts:

- `@RestController`
- `@GetMapping`
- `List.of()`
- JSON array response
- Java record as response model

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise06/StudentListController.java`

Status:

Done

Exercise 07 - Optional query parameter filter

Purpose:

This exercise introduces how to use an optional query parameter to filter a JSON list response.

URLs:

`http://localhost:8080/exercise07/students`

`http://localhost:8080/exercise07/students?role=backend`

HTTP method:

GET

Main concepts:

- `@RestController`
- `@GetMapping`
- `@RequestParam(required = false)`
- Optional query parameters
- Filtering with Stream API
- `equalsIgnoreCase()`

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise07/StudentFilterController.java`

Status:

Done

Exercise 08 - Receive JSON with POST

Purpose:

This exercise introduces how Spring Boot can receive JSON data in the request body and convert it into a Java object.

URL:

`http://localhost:8080/exercise08/students`

HTTP method:

POST

Main concepts:

- `@RestController`
- `@PostMapping`
- `@RequestBody`
- JSON request body
- Java record as request model
- Java record as response model

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise08/StudentPostController.java`

Status:

Done

Exercise 09 - ResponseEntity with POST

Purpose:

This exercise introduces how to return an HTTP status code together with a JSON response using `ResponseEntity`.

URL:

http://localhost:8080/exercise09/students

HTTP method:

POST

Main concepts:

- `@RestController`
- `@PostMapping`
- `@RequestBody`
- `ResponseEntity`
- `HttpStatus.CREATED`
- HTTP 201 Created

File:

src/main/java/com/angel/springbootlearning/exercises/exercise09/StudentResponseEntityController.java

Status:

Done

Exercise 10 - Basic validation with POST

Purpose:

This exercise introduces basic manual validation before creating a response. If the received JSON contains invalid data, the controller returns an HTTP 400 Bad Request response.

URL:

http://localhost:8080/exercise10/students

HTTP method:

POST

Main concepts:

- `@RestController`
- `@PostMapping`
- `@RequestBody`
- `ResponseEntity`
- `HttpStatus.CREATED`
- `HttpStatus.BAD_REQUEST`
- Manual validation
- Error response model

File:

src/main/java/com/angel/springbootlearning/exercises/exercise10/StudentValidationController.java

Status:

Done

Exercise 11 - Reusable validation with private methods

Purpose: This exercise improves the previous manual validation by moving repeated validation logic into private helper methods.

URL: <http://localhost:8080/exercise11/students>

HTTP method: POST

Main concepts:

- Manual validation
- Private helper methods
- `ResponseEntity<?>`
- `HttpStatus.CREATED`
- `HttpStatus.BAD_REQUEST`
- Custom error response
- Cleaner controller logic
- Reading `400 Bad Request` responses in PowerShell

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise11/StudentReusableValidationController.java`

PowerShell valid request:

```
$body = @{"name" = "Angel"; "role" = "backend"} | ConvertTo-Json

Invoke-RestMethod `
  -Uri "http://localhost:8080/exercise11/students" `
  -Method Post `
  -ContentType "application/json" `
  -Body $body
```

PowerShell invalid request:

```
$body = @{"name" = ""; "role" = "backend"} | ConvertTo-Json

try {
  Invoke-WebRequest `
```

```
-Uri "http://localhost:8080/exercise11/students" `
-Method Post `
-ContentType "application/json" `
-Body $body
}
catch {
    $reader = New-Object
    System.IO.StreamReader($_.Exception.Response.GetResponseStream())
    $reader.ReadToEnd()
}
```

Expected error response:

```
{
  "message": "The student name is required!"
}
```

Status: Done

Exercise 12 - Validation of several fields

Purpose: This exercise extends manual validation by checking several fields before creating a response: name, role, age and email.

URL: http://localhost:8080/exercise12/students

HTTP method: POST

Main concepts:

- Manual validation
- Validation of several fields
- Private helper methods
- `ResponseEntity<?>`
- `HttpStatus.CREATED`
- `HttpStatus.BAD_REQUEST`
- Custom error response
- Basic email validation
- Numeric range validation
- Cleaner controller logic

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise12/StudentSeveralFieldsValidationController.java`

Tested cases:

- Valid request returns HTTP 201 Created.
- Empty name returns HTTP 400 Bad Request.

- Empty role returns HTTP 400 Bad Request.
- Invalid age returns HTTP 400 Bad Request.
- Invalid email returns HTTP 400 Bad Request.

Status: Done

Exercise 13 - Structured error responses

Purpose: This exercise improves error handling by returning a more structured error response. Instead of returning only a message, the API returns the HTTP status code, the HTTP error name and a custom validation message.

URL: `http://localhost:8080/exercise13/students`

HTTP method: POST

Main concepts:

- Manual validation
- Structured error response
- `ResponseEntity<?>`
- `HttpStatus.BAD_REQUEST`
- `status.value()`
- `status.getReasonPhrase()`
- Custom `ErrorResponse`
- Basic regex validation for email

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise13/StudentStructuredErrorController.java`

Status: Done

Exercise 14 - HTTP 404 Not Found

Purpose: This exercise introduces how to return an HTTP 404 Not Found response when a requested resource does not exist.

URL: `http://localhost:8080/exercise14/students/{id}`

HTTP method: GET

Main concepts:

- `@GetMapping`
- `@PathVariable`
- `ResponseEntity<?>`
- HTTP 200 OK
- HTTP 404 Not Found
- Simulated resource lookup

- Structured error response

File:

src/main/java/com/angel/springbootlearning/exercises/exercise14/StudentNotFoundController.java

Status: Done

Exercise 15 - HTTP 204 No Content

Purpose: This exercise introduces how to return an HTTP 204 No Content response when an operation is completed successfully but does not need to return a response body.

URL: http://localhost:8080/exercise15/students/{id}

HTTP method: DELETE

Main concepts:

- `@DeleteMapping`
- `@PathVariable`
- `ResponseEntity<?>`
- HTTP 204 No Content
- HTTP 404 Not Found
- Simulated delete operation
- Response without body

File:

src/main/java/com/angel/springbootlearning/exercises/exercise15/StudentNoContentController.java

Status: Done

Exercise 16 - HTTP 409 Conflict

Purpose: This exercise introduces how to return an HTTP 409 Conflict response when the request is valid but conflicts with existing data.

URL: http://localhost:8080/exercise16/students

HTTP method: POST

Main concepts:

- `@PostMapping`
- `@RequestBody`
- `ResponseEntity<?>`
- HTTP 201 Created
- HTTP 400 Bad Request
- HTTP 409 Conflict

- Duplicate value validation
- Simulated existing data

File:

src/main/java/com/angel/springbootlearning/exercises/exercise16/StudentConflictController.java

Status: Done

Exercise 17 - HTTP 422 Unprocessable Content

Purpose: This exercise introduces how to return an HTTP 422 response when the received JSON is valid but does not follow a business rule.

URL: http://localhost:8080/exercise17/students

HTTP method: POST

Main concepts:

- `@PostMapping`
- `@RequestBody`
- Manual validation
- Business rule validation
- `HttpStatus.UNPROCESSABLE_CONTENT`
- HTTP 422 Unprocessable Content
- Allowed values validation
- Structured error response

File:

src/main/java/com/angel/springbootlearning/exercises/exercise17/StudentUnprocessableEntityController.java

Status: Done

Exercise 18 - Common HTTP status codes

Purpose: This exercise consolidates several common HTTP status codes used in REST APIs in a single controller.

URL: http://localhost:8080/exercise18/students

HTTP methods: GET, POST, DELETE

Main concepts:

- `ResponseEntity`
- HTTP 200 OK
- HTTP 201 Created
- HTTP 204 No Content

- HTTP 400 Bad Request
- HTTP 404 Not Found
- HTTP 409 Conflict
- `@GetMapping`
- `@PostMapping`
- `@DeleteMapping`
- `@PathVariable`
- `@RequestBody`
- Structured error responses
- Simulated student data

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise18/StudentHttpStatusController.java`

Tested cases:

- GET all students returns HTTP 200 OK.
- GET existing student by id returns HTTP 200 OK.
- GET missing student by id returns HTTP 404 Not Found.
- POST valid student returns HTTP 201 Created.
- POST empty required field returns HTTP 400 Bad Request.
- POST duplicated student name returns HTTP 409 Conflict.
- DELETE existing student returns HTTP 204 No Content.
- DELETE missing student returns HTTP 404 Not Found.

Status: Done

Exercise 19 - Conditional validation

Purpose: This exercise introduces conditional validation. Some fields are required only when another field has a specific value.

URL: `http://localhost:8080/exercise19/students`

HTTP method: POST

Main concepts:

- Manual validation
- Conditional validation
- Business rule validation
- `ResponseEntity<?>`
- `HttpStatus.BAD_REQUEST`
- `HttpStatus.UNPROCESSABLE_CONTENT`
- HTTP 422 Unprocessable Content
- Private helper methods
- Structured error response

File:

src/main/java/com/angel/springbootlearning/exercises/exercise19/StudentConditionalValidationController.java

Tested cases:

- Backend student without portfolio URL returns HTTP 201 Created.
- Frontend student without portfolio URL returns HTTP 422 Unprocessable Content.
- Fullstack student without portfolio URL returns HTTP 422 Unprocessable Content.
- Frontend student with portfolio URL returns HTTP 201 Created.
- Empty name returns HTTP 400 Bad Request.
- Empty role returns HTTP 400 Bad Request.
- Invalid role returns HTTP 422 Unprocessable Content.

Status: Done

Exercise 20 - Refactor responses

Purpose: This exercise refactors response creation by using private helper methods for both successful and error responses.

URL: http://localhost:8080/exercise20/students

HTTP method: POST

Main concepts:

- Response refactoring
- Private helper methods
- `ResponseEntity<?>`
- `ResponseEntity<StudentResponse>`
- `ResponseEntity<ErrorResponse>`
- `HttpStatus.CREATED`
- `HttpStatus.BAD_REQUEST`
- `HttpStatus.UNPROCESSABLE_CONTENT`
- Reusable success response
- Reusable error response
- Cleaner controller logic

File:

src/main/java/com/angel/springbootlearning/exercises/exercise20/StudentResponseRefactorController.java

Tested cases:

- Valid request returns HTTP 201 Created.
- Empty name returns HTTP 400 Bad Request.
- Empty role returns HTTP 400 Bad Request.
- Invalid role returns HTTP 422 Unprocessable Content.

Status: Done

Exercise 21 - Mutable in-memory list

Purpose: This exercise introduces an `ArrayList` as temporary mutable storage. The stored students remain available only while the application is running.

URL: `http://localhost:8080/exercise21/students`

HTTP method: GET

Main concepts:

- `ArrayList`
- `List`
- Mutable in-memory storage
- `@RequestMapping`
- `@GetMapping`
- JSON array response
- Temporary data storage

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise21/StudentListController.java`

PowerShell request:

```
curl.exe -s "http://localhost:8080/exercise21/students"
```

Expected response:

```
[  
  "Ángel",  
  "Kratos"  
]
```

Tested cases:

- GET request returns HTTP 200 OK.
- The endpoint returns every student stored in the `ArrayList`.
- The stored data disappears when the application is restarted.

Status: Done

Exercise 22 - Create a student

Purpose: This exercise receives a student as JSON through a POST request and stores the received object temporarily in an `ArrayList`.

URL: `http://localhost:8080/exercise22/students`

HTTP method: POST

Main concepts:

- `ArrayList`
- `@PostMapping`
- `@RequestBody`
- JSON request body
- Java record as data model
- `ResponseEntity`
- `HttpStatus.CREATED`
- HTTP 201 Created
- Temporary in-memory storage

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise22/StudentCreateController.java`

PowerShell request:

```
$body = @{
    id    = 1
    name  = "Angel"
    role  = "Backend developer"
} | ConvertTo-Json

Invoke-RestMethod `
    -Uri "http://localhost:8080/exercise22/students" `
    -Method Post `
    -ContentType "application/json" `
    -Body $body
```

Expected response:

```
id name  role
-- ----  ---
1 Angel Backend developer
```

Tested cases:

- A valid JSON request returns HTTP 201 Created.
- The received student is added to the in-memory list.
- The created student is returned in the response body.

Status: Done

Exercise 23 - List stored students

Purpose: This exercise stores students temporarily in an `ArrayList` and returns all stored students through a GET request.

URL: `http://localhost:8080/exercise23/students`

HTTP methods: GET, POST

Main concepts:

- `ArrayList`
- Mutable in-memory storage
- `@PostMapping`
- `@GetMapping`
- `@RequestBody`
- `ResponseEntity`
- HTTP 201 Created
- JSON list response
- Creating and listing resources
- Multiple PowerShell requests with `ForEach-Object`

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise23/Exercise23StudentListController.java`

PowerShell create requests:

```
$students = @(
    @{
        id    = 1
        name  = "Angel"
        role  = "Backend developer"
    },
    @{
        id    = 2
        name  = "Kratos"
        role  = "God of War"
    }
)

$students | ForEach-Object {
    Invoke-RestMethod `
        -Uri "http://localhost:8080/exercise23/students" `
        -Method Post `
        -ContentType "application/json" `
        -Body ($_ | ConvertTo-Json)
}
```


PowerShell list request:

```
curl.exe -s "http://localhost:8080/exercise23/students"
```

Expected response:

```
[
  {
    "id": 1,
    "name": "Angel",
    "role": "Backend developer"
  },
  {
    "id": 2,
    "name": "Kratos",
    "role": "God of War"
  }
]
```

Tested cases:

- POST request stores a student and returns HTTP 201 Created.
- Several students can be created using several POST requests.
- GET request returns HTTP 200 OK.
- GET request returns every student currently stored in memory.
- A Spring bean name conflict was resolved by giving the controller a unique class name.

Status: Done

Exercise 24 - Find a student by id

Purpose: This exercise searches for a specific student inside an in-memory `ArrayList` using the id received as a path variable.

URL: `http://localhost:8080/exercise24/students/{id}`

HTTP method: GET

Main concepts:

- `ArrayList`
- `@GetMapping`
- `@PathVariable`
- Dynamic resource lookup
- Iterating through a list
- Comparing resource identifiers

- `ResponseEntity`
- HTTP 200 OK
- HTTP 404 Not Found
- Response without body

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise24/StudentSearchController.java`

PowerShell successful request:

```
Invoke-RestMethod `
  -Uri "http://localhost:8080/exercise24/students/2" `
  -Method Get
```

Expected response:

```
id name    role
-- ----    -
2  Kratos  God of War
```

PowerShell missing student request:

```
try {
    Invoke-WebRequest `
      -Uri "http://localhost:8080/exercise24/students/99" `
      -Method Get `
      -UseBasicParsing
}
catch {
    $_.Exception.Response.StatusCode.value__
}
```

Expected error status:

```
404
```

Tested cases:

- Existing student id returns HTTP 200 OK.
- Existing student id returns the matching student as JSON.
- Missing student id returns HTTP 404 Not Found.
- The search iterates through the in-memory list until a matching id is found.

Status: Done

Exercise 25 - Delete a student by id

Purpose: This exercise introduces how to delete a student from an in-memory `ArrayList` using an id received through the URL.

URL: `http://localhost:8080/exercise25/students/{id}`

HTTP method: DELETE

Main concepts:

- `@DeleteMapping`
- `@PathVariable`
- `ArrayList`
- `removeIf()`
- `ResponseEntity<Void>`
- HTTP 204 No Content
- HTTP 404 Not Found
- Resource deletion
- In-memory data management

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise25/StudentDeleteController.java`

PowerShell successful request:

```
$response = Invoke-WebRequest `
    -Uri "http://localhost:8080/exercise25/students/1" `
    -Method Delete `
    -UseBasicParsing

$response.StatusCode
```

Expected status:

204

PowerShell missing student request:

```
try {
    Invoke-WebRequest `
        -Uri "http://localhost:8080/exercise25/students/99" `
        -Method Delete `
        -UseBasicParsing
}
```

```
}  
catch {  
    $_.Exception.Response.StatusCode.value__  
}
```

Expected error status:

404

Tested cases:

- Existing student id returns HTTP 204 No Content.
- The matching student is removed from the in-memory list.
- Missing student id returns HTTP 404 Not Found.
- Repeating the deletion of the same student returns HTTP 404 Not Found.
- The deleted student is restored when the application is restarted because the sample data is created in the controller constructor.

Status: Done

Exercise 26 - Complete student update

Purpose: This exercise introduces how to completely replace the data of an existing student using a PUT request and an id received through the URL.

URL: `http://localhost:8080/exercise26/students/{id}`

HTTP method: PUT

Main concepts:

- `@PutMapping`
- `@PathVariable`
- `@RequestBody`
- `ArrayList`
- Iterating through a list by index
- `List.get()`
- `List.set()`
- Complete resource replacement
- `ResponseEntity<Student>`
- HTTP 200 OK
- HTTP 404 Not Found
- HTTP 405 Method Not Allowed

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise26/StudentFullUpdateController.java`

Interactive PowerShell request:

```
$id = Read-Host "Introduce el ID del estudiante"
$name = Read-Host "Introduce el nuevo nombre"
$role = Read-Host "Introduce el nuevo rol"

$body = @{
    id    = [int]$id
    name  = $name
    role  = $role
} | ConvertTo-Json

try {
    $student = Invoke-RestMethod `
        -Uri "http://localhost:8080/exercise26/students/$id" `
        -Method Put `
        -ContentType "application/json" `
        -Body $body

    Write-Host "Student updated successfully:"
    $student
}
catch {
    $statusCode = $_.Exception.Response.StatusCode.value__

    if ($statusCode -eq 404) {
        Write-Host "Student not found."
    }
    else {
        Write-Host "Request failed with status code $statusCode."
    }
}
```

Expected successful response:

```
{
  "id": 1,
  "name": "Angel Plata",
  "role": "Java developer"
}
```

Expected missing student response:

```
Student not found.
```

Tested cases:

- Existing student id returns HTTP 200 OK.
- All student fields are replaced with the received data.
- The id from the URL is preserved in the updated student.
- Missing student id returns HTTP 404 Not Found.
- Sending a POST request instead of PUT returns HTTP 405 Method Not Allowed.
- `List.set(index, updatedStudent)` replaces the student stored at a specific list position.

Status: Done

Exercise 27 - Partial student update

Purpose: This exercise introduces how to update only the fields received in a PATCH request while preserving the existing values of the other student fields.

URL: `http://localhost:8080/exercise27/students/{id}`

HTTP method: PATCH

Main concepts:

- `@PatchMapping`
- `@PathVariable`
- `@RequestBody`
- Partial resource update
- Optional request fields
- `ArrayList`
- `List.get()`
- `List.set()`
- Ternary operator
- Null validation
- `isBlank()`
- HTTP 200 OK
- HTTP 404 Not Found
- REST Client `.http` files

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise27/StudentPartialUpdateController.java`

Request file: `requests/exercise27.http`

Update only the name:

```
### Update only the student's name
PATCH http://localhost:8080/exercise27/students/1
Content-Type: application/json

{
```

```
"name": "Angel Plata"
}
```

Update only the role:

```
### Update only the student's role
PATCH http://localhost:8080/exercise27/students/1
Content-Type: application/json

{
  "role": "Java backend developer"
}
```

Update both fields:

```
### Update the student's name and role
PATCH http://localhost:8080/exercise27/students/2
Content-Type: application/json

{
  "name": "Kratos of Sparta",
  "role": "Backend destroyer"
}
```

Send an empty body:

```
### Preserve every current value
PATCH http://localhost:8080/exercise27/students/1
Content-Type: application/json

{
}
```

Try to update a missing student:

```
### Try to update a missing student
PATCH http://localhost:8080/exercise27/students/99
Content-Type: application/json

{
  "role": "Developer"
}
```

Expected successful response:

```
{
  "id": 1,
  "name": "Angel",
  "role": "Java backend developer"
}
```

Expected missing student status:

404 Not Found

Java language tip:

The ternary operator provides a compact alternative to an `if/else` assignment:

```
String updatedRole =
    request.role() != null && !request.role().isBlank()
        ? request.role()
        : currentStudent.role();
```

It is equivalent to:

```
String updatedRole;

if (request.role() != null && !request.role().isBlank()) {
    updatedRole = request.role();
} else {
    updatedRole = currentStudent.role();
}
```

Tested cases:

- Updating only the name preserves the current role.
- Updating only the role preserves the current name.
- Sending both fields updates both values.
- Sending an empty JSON object preserves every current value.
- Missing student id returns HTTP 404 Not Found.
- The role condition must use `!request.role().isBlank()` so that non-empty roles are accepted.
- The controller does not include a GET endpoint for listing all students.

Status: Done

Exercise 28 - Automatic student id

Purpose: This exercise introduces automatic id generation when creating students. The client sends only the name and role, while the server assigns the next available id.

URL: `http://localhost:8080/exercise28/students`

HTTP methods: GET, POST

Main concepts:

- Automatic id generation
- Simulated autoincrement
- `ArrayList`
- `List.add()`
- `@PostMapping`
- `@GetMapping`
- `@RequestBody`
- `ResponseEntity<Student>`
- `HttpStatus.CREATED`
- Postincrement operator `nextId++`
- Separate request and response models

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise28/StudentAutoIdController.java`

Request file: `requests/exercise28.http`

Tested cases:

- POST creates a student and returns HTTP 201 Created.
- The client does not send the student id.
- The first student receives id `1`.
- Each new student receives the next available id.
- The created student is stored in the `ArrayList`.
- GET returns all stored students.
- Restarting the application clears the list and resets the id counter.

Status: Done

Exercise 29 - Prevent duplicate students

Purpose: This exercise prevents creating students with duplicate names by checking the existing in-memory list before saving.

URL: `http://localhost:8080/exercise29/students`

HTTP methods: GET, POST

Main concepts:

- Duplicate detection

- Stream
- anyMatch()
- equalsIgnoreCase()
- @PostMapping
- @GetMapping
- ResponseEntity<?>
- HttpStatus.CREATED
- HttpStatus.CONFLICT
- HTTP 409 Conflict
- Automatic id generation

File:

src/main/java/com/angel/springbootlearning/exercises/exercise29/StudentDuplicateController.java

Request file: requests/exercise29.http

Tested cases:

- A new student returns HTTP 201 Created.
- A duplicated name returns HTTP 409 Conflict.
- Duplicate detection ignores uppercase and lowercase differences.
- A different student name is created successfully.
- GET returns all stored students.
- A rejected duplicate is not added to the list.

Status: Done

Exercise 30 - Complete student CRUD

Purpose: This exercise integrates all CRUD operations into a single in-memory student controller.

URL: http://localhost:8080/exercise30/students

HTTP methods: GET, POST, PUT, PATCH, DELETE

Main concepts:

- Complete CRUD
- Automatic id generation
- Duplicate prevention
- Manual validation
- @GetMapping
- @PostMapping
- @PutMapping
- @PatchMapping
- @DeleteMapping
- List.get()
- List.add()
- List.set()

- `List.remove()`
- Private helper methods
- HTTP 200, 201, 204, 400, 404 and 409

File:

`src/main/java/com/angel/springbootlearning/exercises/exercise30/StudentCrudController.java`

Request file: `requests/exercise30.http`

Tested cases:

- POST creates students with automatic ids.
- Duplicate names return HTTP 409 Conflict.
- GET returns all students.
- GET by id returns one student or HTTP 404.
- PUT replaces all student data.
- PATCH updates only the received fields.
- DELETE removes a student and returns HTTP 204.
- Invalid POST or PUT data returns HTTP 400.
- Missing resources return HTTP 404.

Status: Done